STRUCTURES

Introduction

- Our variables until now
  - Single variable

    int i, char c, float f, double d
  - Set of same type elements: Array

    int a[10], char c[20]

Introduction

- If data are not same type, but related? Example: Information about you!
  - Name
  - Age
  - Reg.Number
  - Grade
  - GPA

Introduction

- If data are not same type, but related? Example: Information about you!
  - Name (char[])
  - Age (int)
  - Reg.Number (char[])
  - Grade (char)
  - GPA (double)

Introduction

- So, how to save this student information?
- Option 1:
  - Use separate variables
    - char st1_name[20];
    - int st1_age;
    - char st1_regNo[10];
    - char st1_grade;
    - double st1_gpa;
- Option 2:
  - Put them all together in one data unit
    - called struct

Structure Basics

- A structure is a collection of data values, called *data members*, that form a single unit.
- Unlike arrays, the data members can be of different types.

Struct in C: Method 1

```
struct {
        <variable declaration>;
} <identifier list>;
```

Example:

```
struct {
        char name[20];
        char regNo[10];
        int age;
        double gpa;
} st1;
```

- We declare a variable st1
- Type of st1 is struct
- age is a member of the struct
- gpa is a member of the struct

Example:

```
struct {
        char name[20];
        char regNo[10];
        int age;
        double gpa;
} st1, st2, st3;
```

- We declare three variables: st1, st2, st3
- Type of st1, st2, st3 is the struct
- In this method, we cannot reuse the struct definition in other location (e.g., input of function)

Struct in C: Method 2

```
struct <tag> {
        <variable declaration>;
};
```

Struct <tag> <identifiers>;

Example:

```
struct student {
        char name[20];
        char regNo[10];
        int age;
        double gpa;
} ;
struct student st1, st2, st3;
```

- We define a struct with tag student
  - We do not allocate the memory. It is just a definition
- We declare variable st1, st2 and st3 from student

Declaring Structures

```
struct {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
} st1, st2, st3;
```

```
struct student {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
};
struct student st1, st2, st3;
```

- The name “student" is called a structure tag
- Variables declared within the braces of the structure definition are the structure’s members

Declaring Structures

- Members of the same structure type must have unique names, but two different structure types may contain members of the same name without conflict

Declaring Structures

- Members of the same structure type must have unique names, but two different structure types may contain members of the same name without conflict

```
struct employee {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
};

struct student {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
};
```

```
struct student {
    char name[20];
    char regNo[10];
    int name;
    double gpa;
};
```

Structures as New Data Type

- When we define a new struct, in fact we are define a new data type
  - Then we use the new data type and define variables

- So, we need to learn how to work with it
  - Access to members
  - Operators for struct
  - Pointer to struct
  - struct in functions
  - Array of struct

Size of struct

```
struct {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
} st1, st2, st3;
```

- sizeof(struct student) -> ?
- sizeof(st1) -> ?

Size of struct

- When we create an object of some structure, then compiler allocate contiguous memory for the data members of the structure.
- The size of allocated memory is at least the sum of sizes of all data members.
- The compiler can use padding and in that case there is will be unused space created between two data members.
- The padding is done for the alignment of data member which makes the access to the member faster.

Size of struct

```
struct {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
} st1, st2, st3;
```

```
printf("size of student: %d\n", sizeof(struct student));
printf("size of sit: %d\n", sizeof(st1));
```

Size of struct

```
struct {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
} st1, st2, st3;
```

```
printf("size of student: %d\n", sizeof(struct student));
printf("size of sit: %d\n", sizeof(st1));
```

```
size of student: 48
size of st1: 48
```

Accessing to the Members of struct

<struct variable name>.<element name>

Accessing to the Members of struct

<struct variable name>.<element name>

```
struct student {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
};

void main() {

    struct student st1;

    st1.age = 21;
    st1.gpa = 3.5;

    printf("Student 1 age: %d\n", st1.age);
    printf("Student 1 gpa: %.1f\n", st1.gpa);
}
```

Accessing to the Members of struct

<struct variable name>.<element name>

```
struct student {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
};

void main() {

    struct student st1;

    st1.age = 21;
    st1.gpa = 3.5;

    printf("Student 1 age: %d\n", st1.age);
    printf("Student 1 gpa: %.1f\n", st1.gpa);
}
```

```
Student 1 age: 41
Student 1 gpa: 3.5
```

Accessing to the Members of struct

IDEs shows you the variables assigned to the struct.

```
struct student {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
};

void main() {

    struct student st1;

    st1.
```

variable	int age
variable	double gpa
variable	char name
variable	char regNo

struct Initialization

- Similar to array initialization.

```
struct student {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
};

void main() {

    struct student st1 = {"Vidura Nishantha", "20/CS/012", 24, 3.7};
```

  - “Vidura Nishantha” is assigned to st1.name
  - “20/CS/012” is assigned to st1.regNo
  - 24 is assigned to st1.age
  - 3.7 is assigned to st1.gpa

struct Initialization

- Similar to array initialization.

```
struct student {
    char name[20];
    char regNo[10];
    int age;
    double gpa;
};

void main() {

    struct student st1 = {"Vidura Nishantha", "20/CS/012", 24, 3.7};
```

  - Order of values should be exactly the order of the members
  - The number of values must be <= the number of members
  - Initial values cannot be assigned in struct definition

Nested Structures

```
struct date_type {
    int dd, mm, yy;
};

struct student {
    char name[20];
    char regNo[10];
    struct date_type birthday;
    double gpa;
};
```

Nested Structures

```
struct student {
    char *name;
    char *regNo;
    struct date_type birthday;
    double gpa;
};

void main() {

    struct student st1 = {"Vidura Nishantha", "20/CS/012", {24, 06, 1998}, 3.7};

    struct student st2;
    st2.name = "Asanka Madhawa";
    st2.regNo = "20/IS/085";
    st2.gpa = 3.8;

    st2.birthday.dd = 31;
    st2.birthday.mm = 8;
    st2.birthday.yy = 1997;
```

Declaring Structures

- A structure cannot contain an instance of itself
- For example, a variable of type struct student cannot be declared in the definition for struct student

```
struct date_type {
    int dd, mm, yy;
};

struct student {
    char *name;
    char *regNo;
    struct date_type birthday;
    double gpa;
    struct student st;
};
```

Declaring Structures

- A pointer to struct student, however, may be included

```
struct date_type {
    int dd, mm, yy;
};

struct student {
    char *name;
    char *regNo;
    struct date_type birthday;
    double gpa;
    struct student *st;
};
```

A structure containing a member that is a pointer to the same structure type is referred to as a **self-referential structure**

Operators on Structure

- Copy and Assignment:

```
struct date {
    int dd, mm, yy;
};

void main() {

    struct date d1 = {19, 07, 2020};
    struct date d2;

    //copy all the data from d1 to d2
    d2 = d1;

    printf("d2 day: %d\n", d2.dd);
    printf("d2 month: %d\n", d2.mm);
    printf("d2 year: %d\n", d2.yy);

}
```

Operators on Structure

- Copy and Assignment:

```
struct date {
    int dd, mm, yy;
};

void main() {

    struct date d1 = {19, 07, 2020};
    struct date d2;

    //copy all the data from d1 to d2
    d2 = d1;

    printf("d2 day: %d\n", d2.dd);
    printf("d2 month: %d\n", d2.mm);
    printf("d2 year: %d\n", d2.yy);

}
```

=

```
d2.dd = d1.dd;
d2.mm = d1.mm;
d2.yy = d1.yy;
```

Operators on Structure

- Copy and Assignment:

```
struct date {
    int dd, mm, yy;
};

void main() {

    struct date d1 = {19, 07, 2020};
    struct date d2;

    //copy all the data from d1 to d2
    d2 = d1;

    printf("d2 day: %d\n", d2.dd);
    printf("d2 month: %d\n", d2.mm);
    printf("d2 year: %d\n", d2.yy);

}
```

```
d2 day: 19
d2 month: 7
d2 year: 2020
```

Operators on Structure

- Copy and Assignment:
  - This can not be done for different struct types even though the element set is same.

```
struct date_type_1 {
    int dd, mm, yy;
};

struct date_type_2 {
    int dd, mm, yy;
};

void main() {

    struct date_type_1 d1 = {19, 07, 2020};
    struct date_type_2 d2;

    d2=d1;
```

Operators on Structure

- Basic mathematical operations:
  - We cannot perform +, -, *, / on struct variables

```
struct date_type_1 {
    int dd, mm, yy;
};

struct date_type_2 {
    int dd, mm, yy;
};

void main() {

    struct date_type_1 d1 = {19, 07, 2020};
    struct date_type_1 d2 = {24, 06, 1997};

    struct date_type_1 d3;
    d3 = d2 - d1;
```

Operators on Structure

- Comparing:
  - We cannot compare struct variables
    - ==, <=, <, >, >= cannot be used for struct

```
struct date_type_1 {
    int dd, mm, yy;
};

struct date_type_2 {
    int dd, mm, yy;
};

void main() {

    struct date_type_1 d1 = {19, 07, 2020};
    struct date_type_1 d2 = {24, 06, 1997};

    if(d1<=d2) {
        printf("d2 is the recent date.");
    }
```

Operators on Structure

- Comparing:
  - However, we can compare the member values in struct

```
struct date_type_1 {
    int dd, mm, yy;
};

struct date_type_2 {
    int dd, mm, yy;
};

void main() {

    struct date_type_1 d1 = {19, 07, 2020};
    struct date_type_1 d2 = {24, 06, 1997};

    if(d1.yy<=d2.yy) {
        printf("d2 is the recent date.");
    }
    else
        printf("d1 is the recent date.");

}
```

Operators on Structure

- Comparing:
  - However, we can compare the member values in struct

```c
struct date_type_1 {
    int dd, mm, yy;
};

struct date_type_2 {
    int dd, mm, yy;
};

void main() {

    struct date_type_1 d1 = {19, 07, 2020};
    struct date_type_1 d2 = {24, 06, 1997};

    if(d1.yy<=d2.yy) {
        printf("d2 is the recent date.");
    }
    else
        printf("d1 is the recent date.");

}
```

```
d1 is the recent date.
```

Arrays of Structures

- struct is a type. So we can define an array of struct

```
struct student {
    char *name;
    char *regNo;
    int age;
    double gpa;

};

void main() {

    struct student students[20];
}
```

Example 1:

```c
struct student {
    char name[20];
    int age;
    double gpa;

};

void main() {

    struct student students[SIZE];
    int i, stNo;

    for(i=0; i<SIZE; i++) {
        printf("Enter student %d name: ", i+1);
        scanf("%[^\n]s", students[i].name);
        printf("Enter student %d age: ", i+1);
        scanf("%d", &students[i].age);
        printf("Enter student %d gpa: ", i+1);
        scanf("%lf", &students[i].gpa);
        printf("you entered student %d details.\n\n", i+1);
        getchar();
    }

    printf("Enter the student number you want to see the details: ");
    scanf("%d", &stNo);

    printf("student %d name: %s\n", stNo, students[stNo-1].name);
    printf("student %d age: %d\n", stNo, students[stNo-1].age);
    printf("student %d gpa: %.1f\n", stNo, students[stNo-1].gpa);

}
```

Example 2: (searching)

```c
struct student {
    char name[20];
    int ID;
    double gpa;
};

void main() {

    struct student students[SIZE];
    int i, stID;

    for(i=0; i<SIZE; i++) {
        printf("Enter student %d name: ", i+1);
        scanf("%[^\n]s", students[i].name);
        printf("Enter student %d ID: ", i+1);
        scanf("%d", &students[i].ID);
        printf("Enter student %d gpa: ", i+1);
        scanf("%lf", &students[i].gpa);
        printf("you entered student %d details.\n\n", i+1);
        getchar();
    }

    printf("Enter the student ID you want to see the details: ");
    scanf("%d", &stID);

    for(i=0; i<SIZE; i++) {
        if(students[i].ID==stID) {
            printf("student name: %s\n", students[i].name);
            printf("student gpa: %.1f\n", students[i].gpa);
        }
    }
}
```

UCSC

Quick overview

- We can define new data type using Structure.
- Syntax of structure definition is:

```
struct <structure tag>
{
    <members type> <member name>;
    ...
}<variable name>, ... ;

struct <structure tag> <variable name>,...;
```